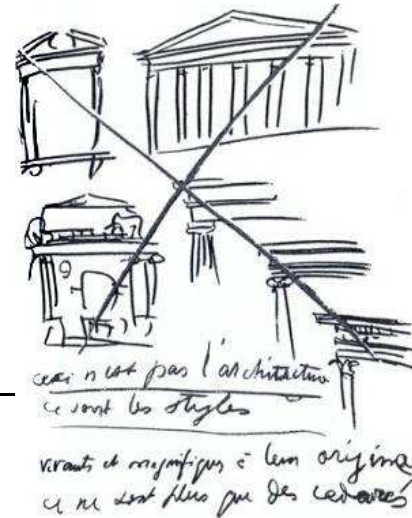


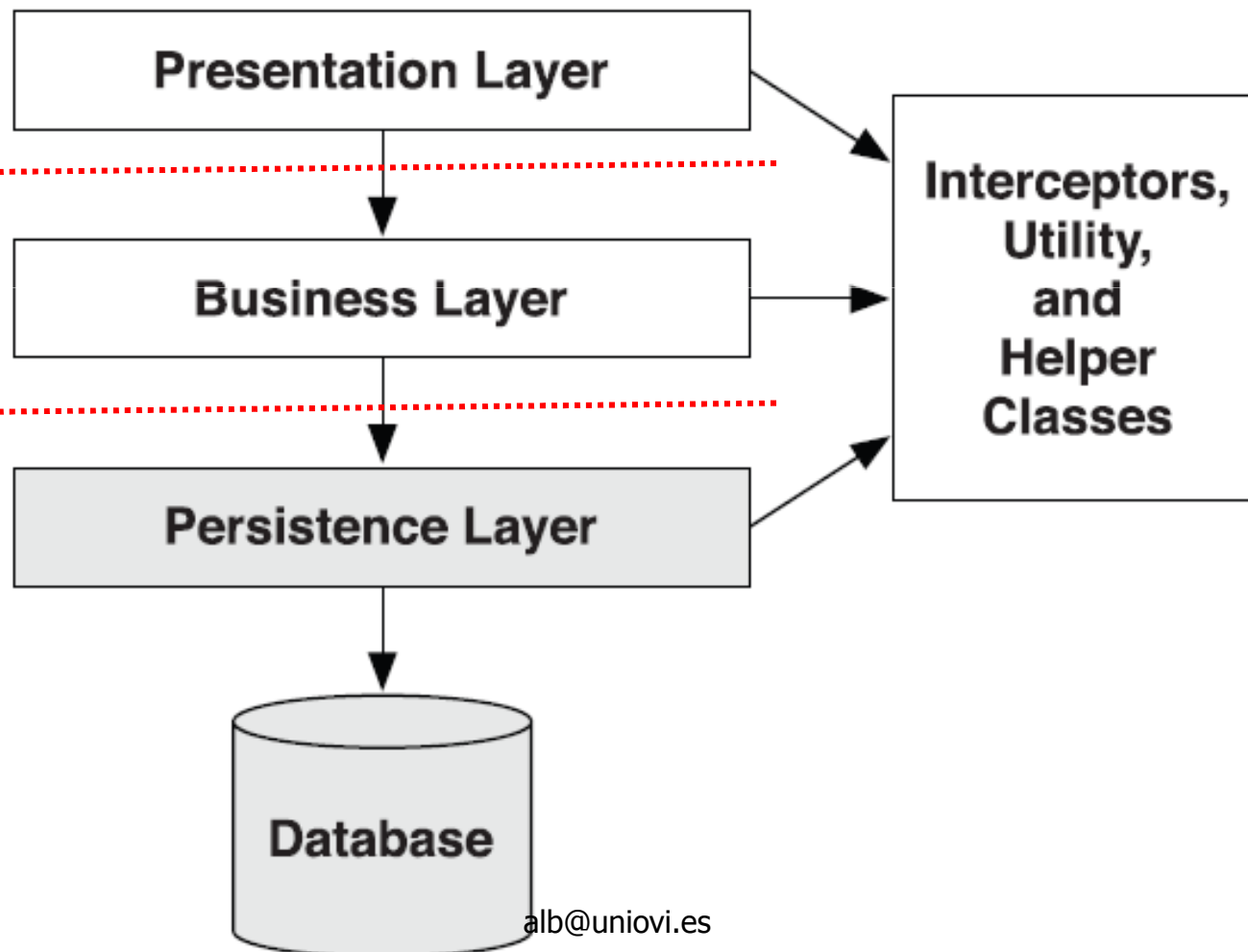
Arquitectura de aplicaciones



Arquitectura en capas

API

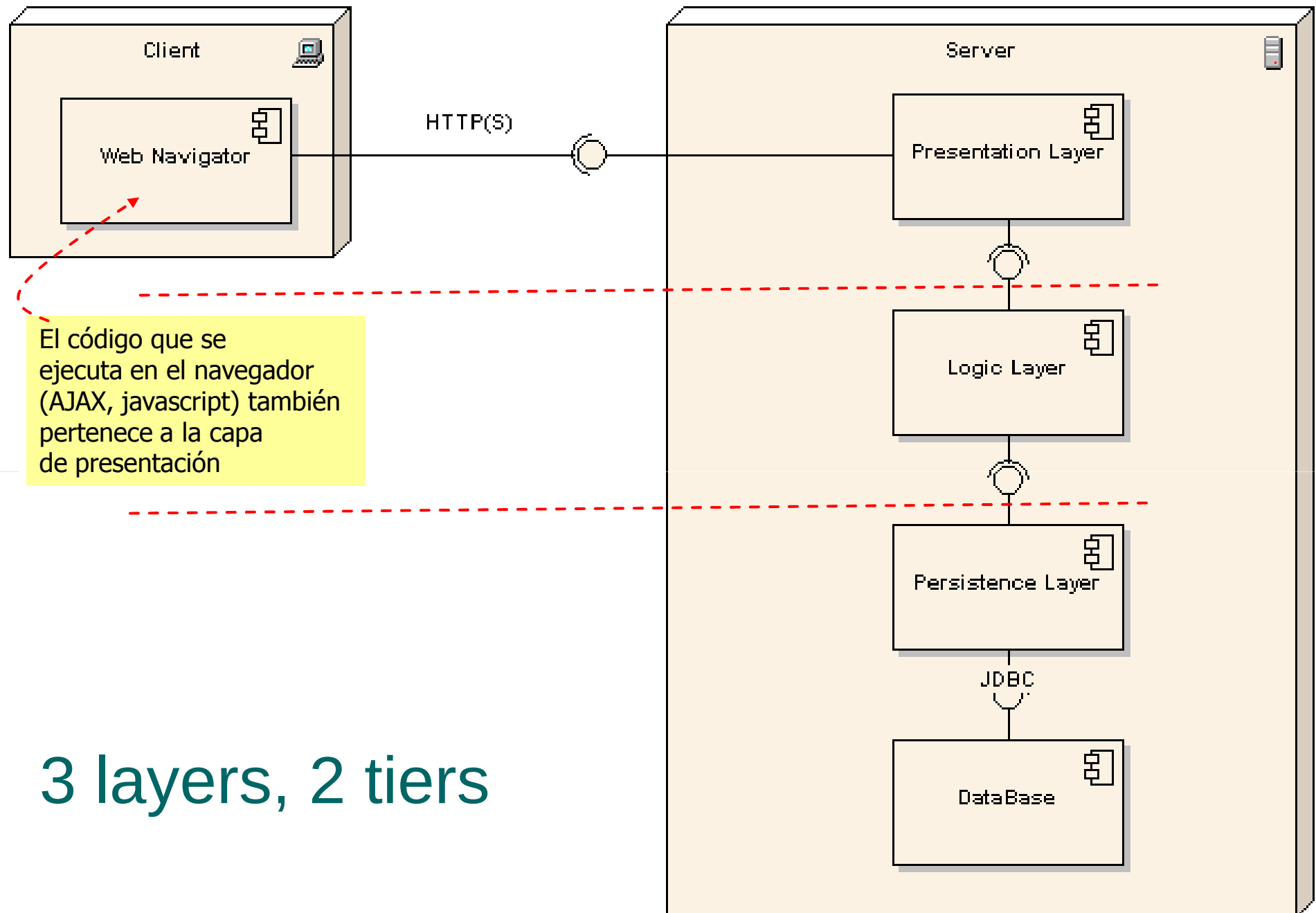
API





Layers y Tiers

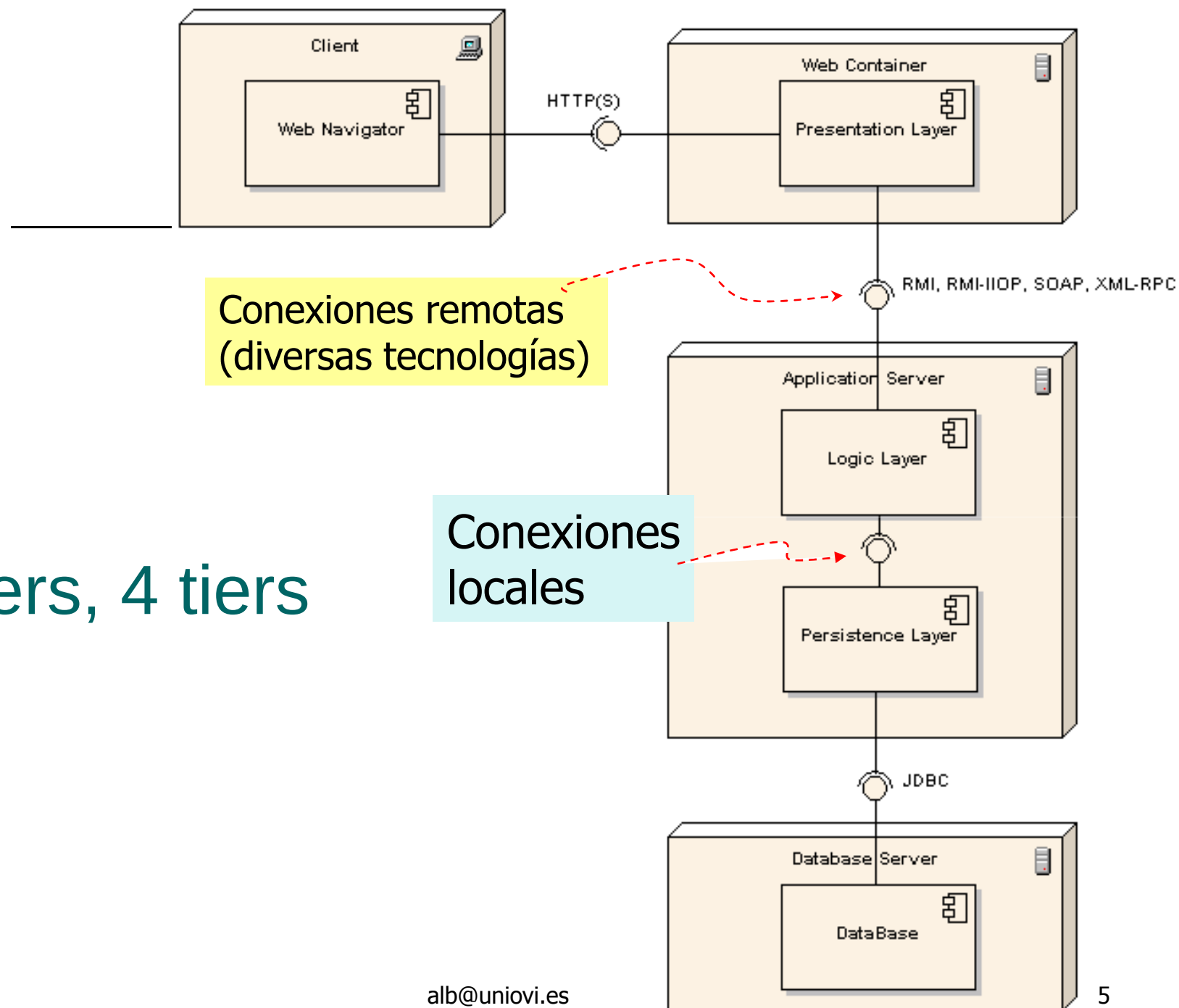
- Layer: capa arquitectónica de la aplicación **software**
 - Presentación, lógica, persistencia
- Tier: capa **física** de la arquitectura de despliegue del hardware
 - Máquinas: Servidor web, servidor de aplicaciones, servidor de base de datos
- Las “**layers**” se despliegan sobre las “**tiers**”

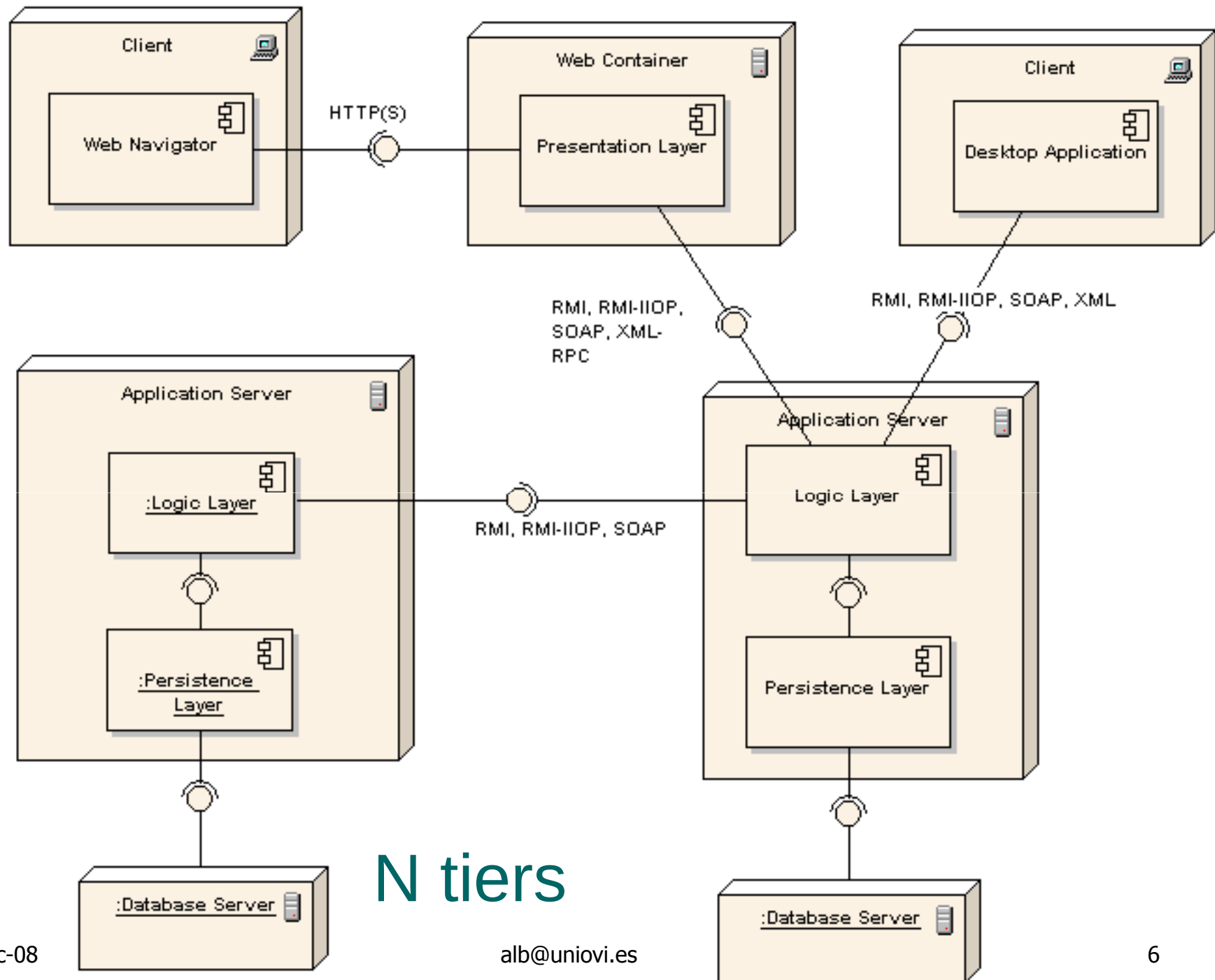


3 layers, 2 tiers



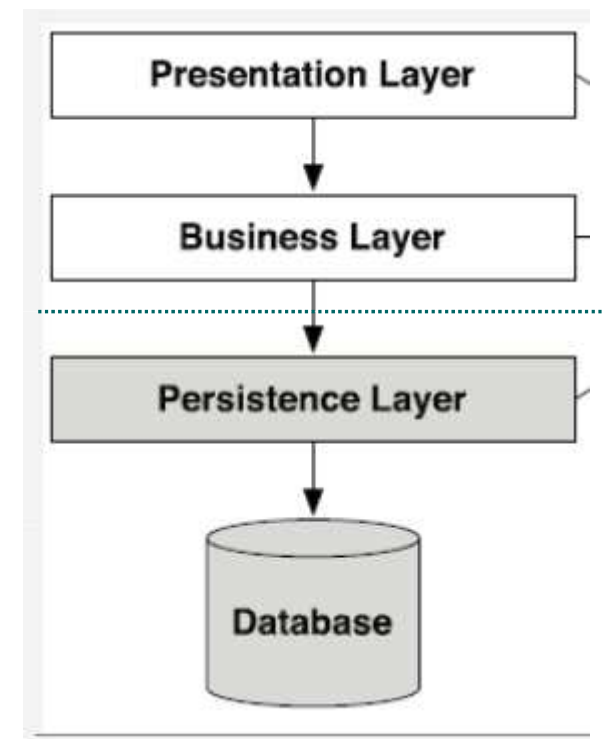
3 layers, 4 tiers



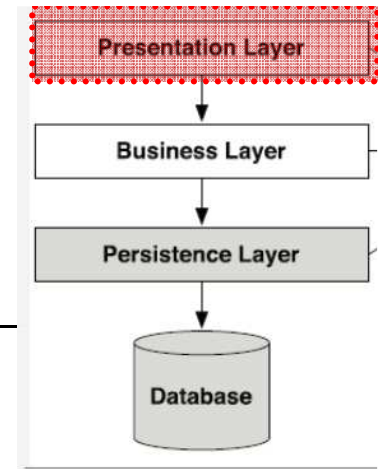


Arquitectura en capas

- Las capas se comunican a través de interfaces
 - Las implementaciones están ocultas al exterior
 - Una factoría sirve una implementación para cada interfaz
 - La capa superior se comunica con la inferior, no al revés
 - Las capas, hechas así, son intercambiables
 - Y según como se hagan reubicables

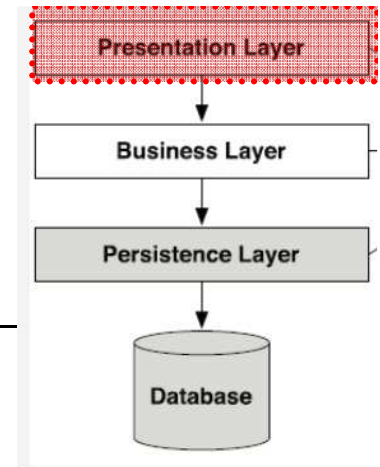


Capa de presentación



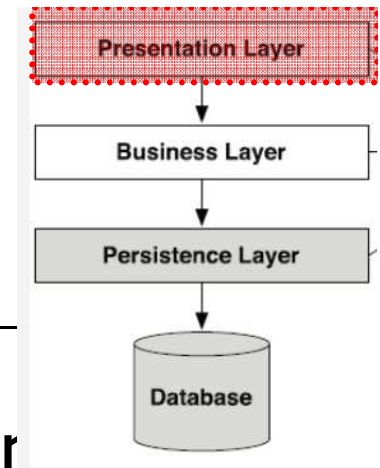
- Resuelve la interacción con el usuario
 - Mostrar datos, formatearlos, ordenarlos
 - Solicitar datos, validarlos
 - Incluye algo de lógica (pero de presentación)
 - Internacionalización
 - Informar de los errores lógicos y de ejecución (errores internos)

Capa de presentación



- Controlar la navegación entre pantallas
- Algunas reglas de negocio pueden ser responsabilidad de esta capa
 - Presentar estos datos así y los otros asá...
 - Ocultar/deshabilitar determinado dato/control si se da tal circunstancia...

Capa de presentación



- Puede estar dividida en subcapas
 - Parte en el servidor (p.e. servidor web)
 - Parte en el cliente (p.e. navegador, AJAX)
- Patrones habituales:
 - MVC → Struts Filter
 - Comando → Struts Actions (xxx.execute())
 - **ServiceLocator** o **Factory** → desacopla la implementación del servicio

Acceso a Lógica: ejemplo

```
public class ViewMatesByEnrollmentAction
    extends ActionSupport
    implements ServletRequestAware {

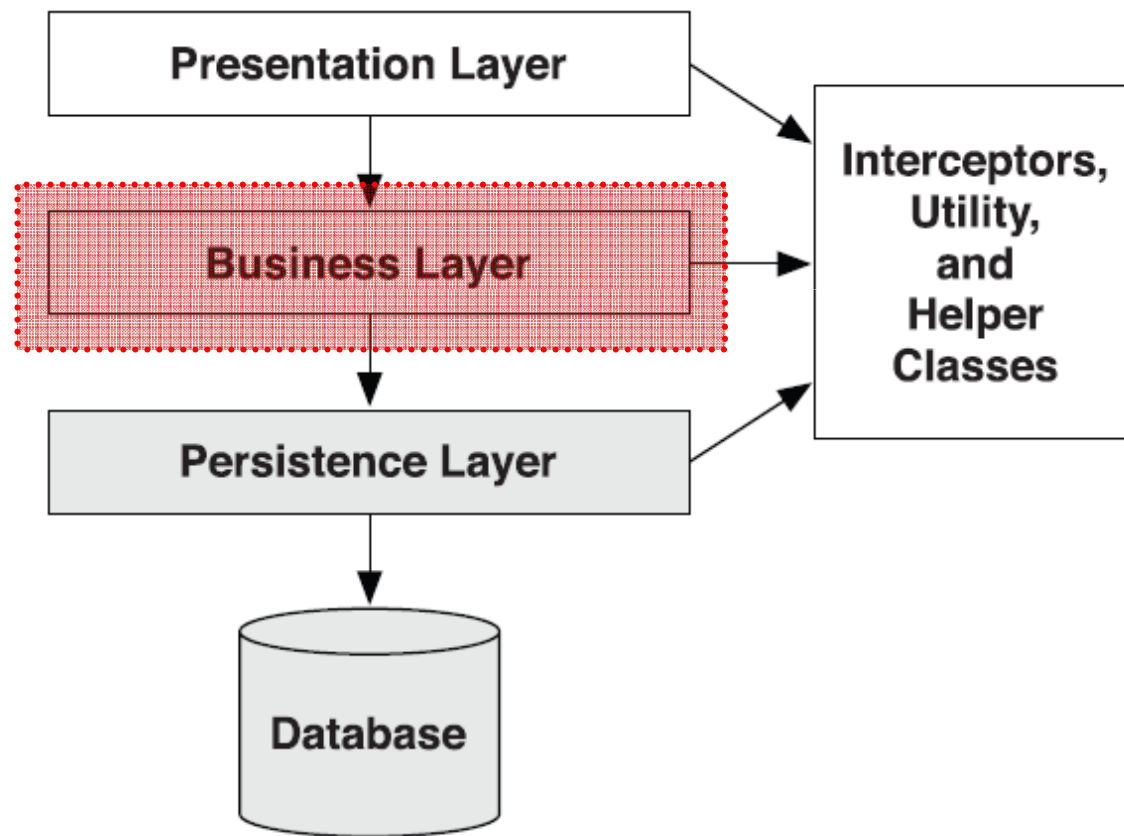
    private HttpServletRequest request;
    private Long enrollmentId;

    public void setServletRequest(HttpServletRequest httpServletRequest) {
        this.request = httpServletRequest;
    }

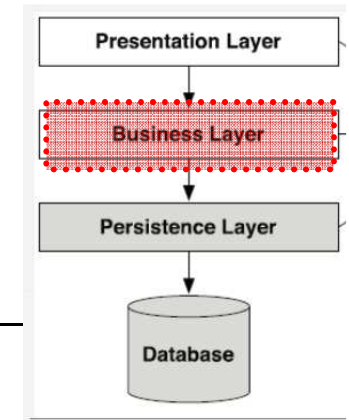
    public String execute() throws Exception {
        if (request.getSession().getAttribute("user") == null) {
            return LOGIN;
        }
        StudentService ss = ServiceLocator.getStudentService();

        request.setAttribute("matesCollection",
            ss.findMatesByEnrollment(enrollmentId));

        return SUCCESS;
    }
}
```

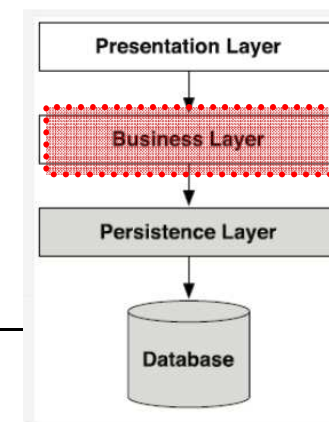


Capa de Negocio: Responsabilidades



- Implementa procesos de negocio identificados durante el análisis funcional.
- Control de acceso a los servicios de negocio desde otras capas.
- Publicación de los servicios de negocio
- Invocación de la capa de persistencia.

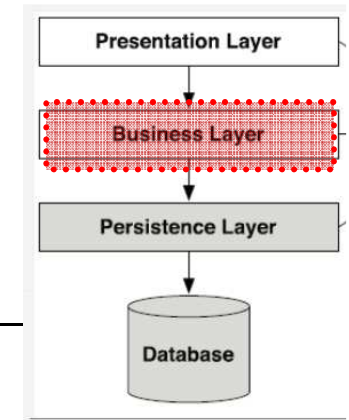
Implementación de Procesos de Negocio



- Independientes de los aspectos de presentación.
- Contra ejemplo:
 - Informe de varias filas donde cada una de ellas deberá sombreadarse de un color dependiendo de un

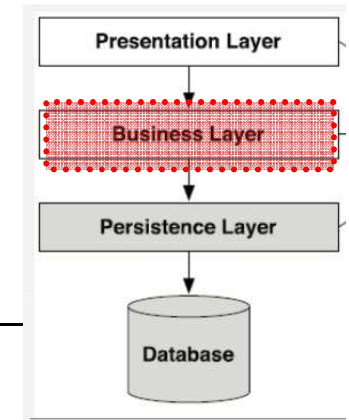
Delegación	2003	2004	Crecimiento
Santander	1.090.004€	1.234.000€	13,21 %
Oviedo	1.245.330€	1.300.320€	4,41 %
Bilbao	1.004.545€	975.034€	-2,93 %

Control de Acceso a Servicios de Negocio



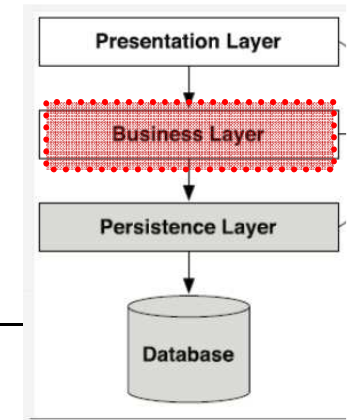
- El control de acceso al servicio de negocio debe hacerse en la capa de negocio, puesto que podemos tener distintas capas de presentación.
- ¿Que perfil puede acceder a un determinado servicio?
 - Se delega en un componente de infraestructura.
- El control se puede hacer a nivel de servicio vertical (cada Façade) o a nivel de método dentro de cada servicio.

Publicación de Servicios de Negocio



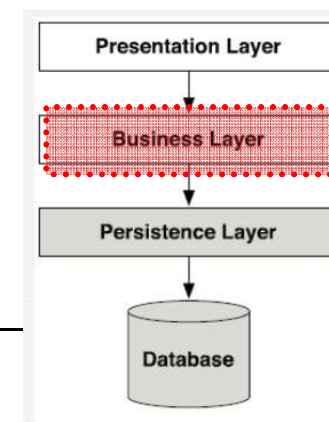
- Hay servicios que se comparten con otros sistemas: Modelo colaborativo.
- La publicación se debe hacer a nivel de la capa de negocio.
- Distintas posibilidades tecnológicas
 - Web Services, RMI, IIOP, RMI-IIOP (EJB), ...
- Nivel de seguridad mayor.

Capa lógica de negocio



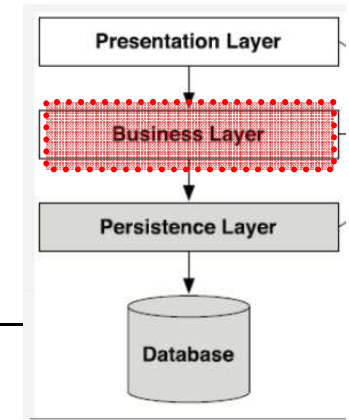
- Ofrece un interfaz de servicios
 - En JEE es una interfaz java
- Cada servicio (método) puede resolver un caso de uso o parte
- Los servicios pueden ser:
 - Sin estado: cada llamada es independiente de las demás; el cliente puede invocar en cualquier orden
 - Con estado: existe noción de sesión, una llamada estará condicionada por las anteriores

Lógica de negocio: implementación



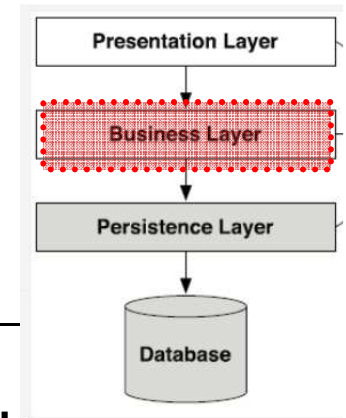
- El cliente sólo conoce la interfaz
 - Habrá una implementación de ese interfaz...
 - ... que puede ser cambiada por otra sin afectar
- Puede estar dividida en subcapas
 - Capa de lógica: es el núcleo central de la aplicación, la esencia del negocio, la lógica y sus reglas
 - Capa de aplicación: añade algún valor al procesamiento de la capa de lógica (p.e. generar un excel, un pdf, importar o exportar datos, etc.)

Capa de lógica: implementación

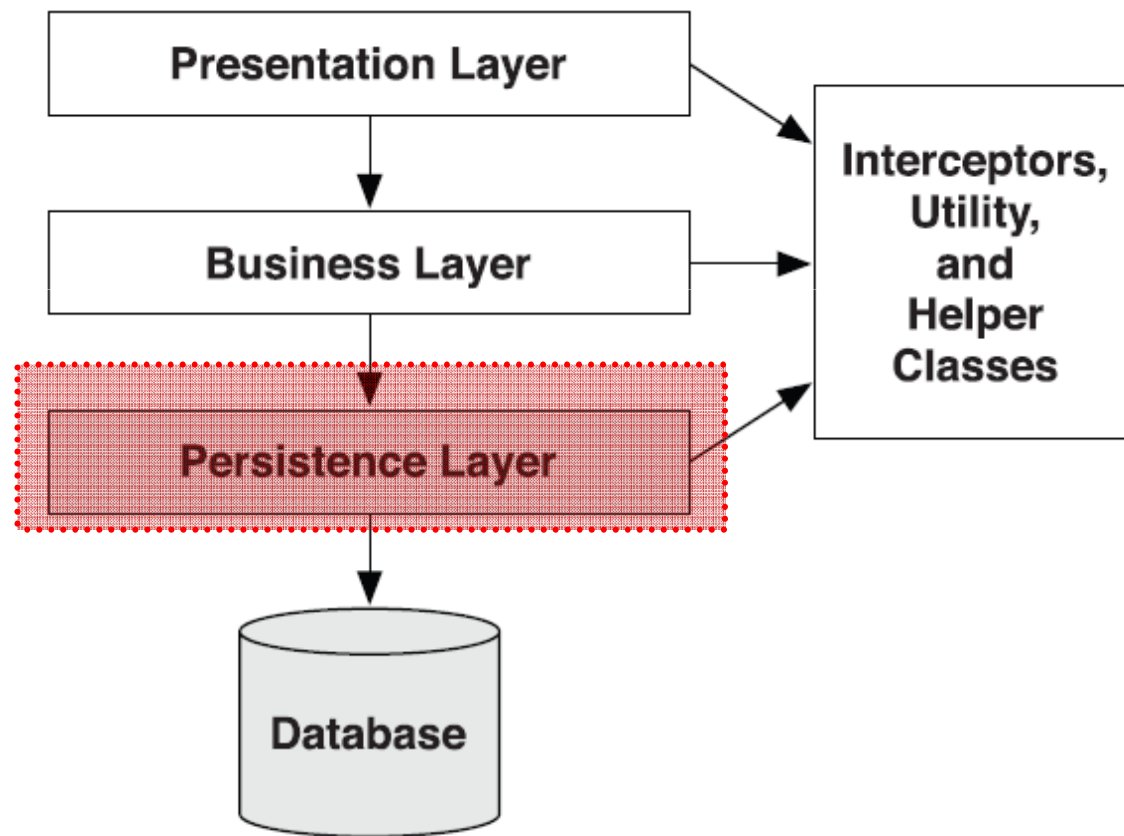


- Varios patrones aplicables:
 - Dominios sencillos: Active record, Record set [fowler]
 - Dominios complejos: Modelo de dominio
 - Problema: gestionar persistencia → mapeador
 - JPA, Hibernate, TopLink, etc
 - Factory
 - Command

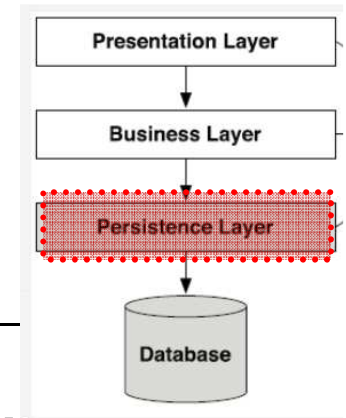
Capa de lógica: implementación



- Si se usa modelo de dominio, compuesta de:
 - Modelo de dominio: incluye lógica pero no toda
 - Clases de proceso sobre el modelo: lógica que no se puede asignar directamente a ninguna clase del modelo de dominio (procesos)
- En esta capa no se debería meter ninguna dependencia de tecnología de infraestructura
 - Debería poderse ejecutar fuera de cualquier entorno (para testear)
 - La persistencia suele ser la principal dependencia. La capa DAO la evita



Capa de persistencia

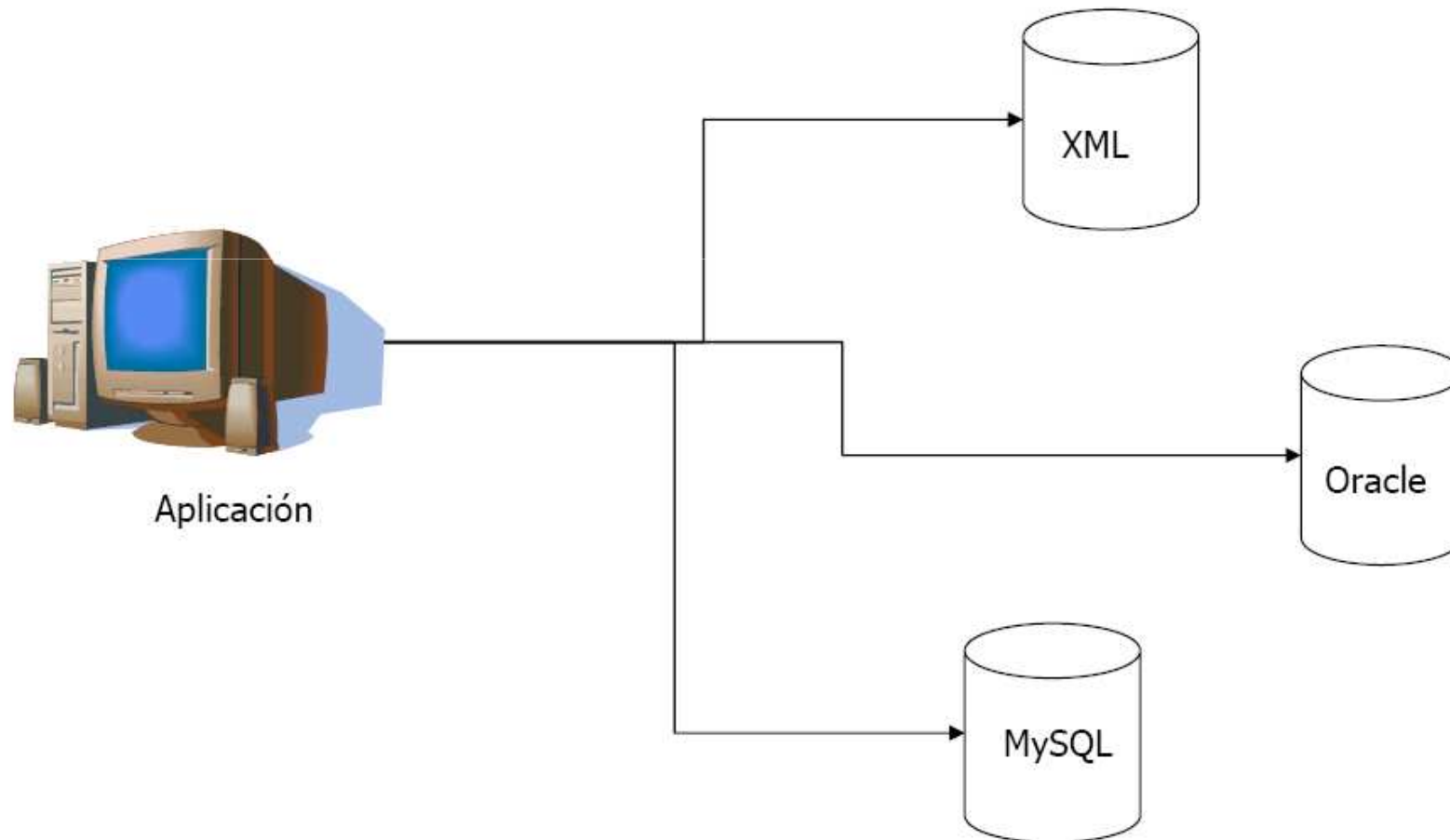
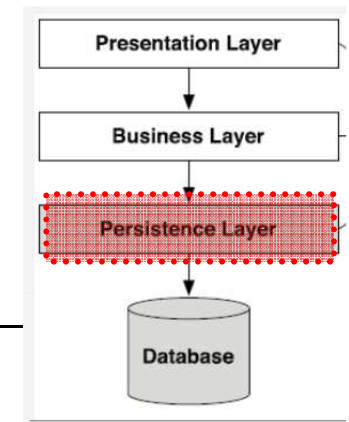


- Ofrece interfaz a la capa superior
- Las distintas implementaciones de la persistencia no deben ser perceptibles por la capa de lógica → independencia
- Uso del patrón DAO
 - Con frecuencia un DAO para cada entidad del modelo
 - Obtenidos a través de una factoría

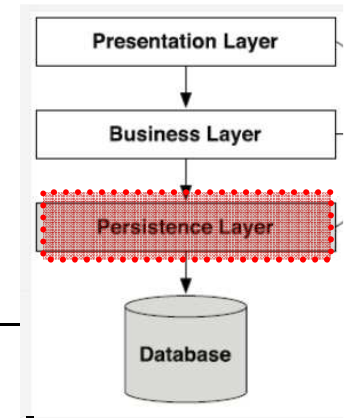
Interfaz de servicio: ejemplo

```
public interface StudentService {  
    Student findById(Long studentId) throws BusinessException;  
  
    Collection<Subject> findEnrolledSubjects(Long studentId, String course)  
        throws BusinessException;  
  
    Collection<Teacher> findTeachersBySubject(Long subjectId)  
        throws BusinessException;  
  
    Collection<Enrollment> findEnrollmentByStudent(Long studentId)  
        throws BusinessException;  
  
    Collection<Assessment> findAssesmentsByEnrollment(Long enrollmentId)  
        throws BusinessException;  
  
    Collection<Student> findMatesByEnrollment(Long enrollmentId)  
        throws BusinessException;  
  
    Collection<Student> findMatesByNameSurnameCourse(String name,  
        String surname, String course) throws BusinessException;  
}
```

Patrón DAO: problema

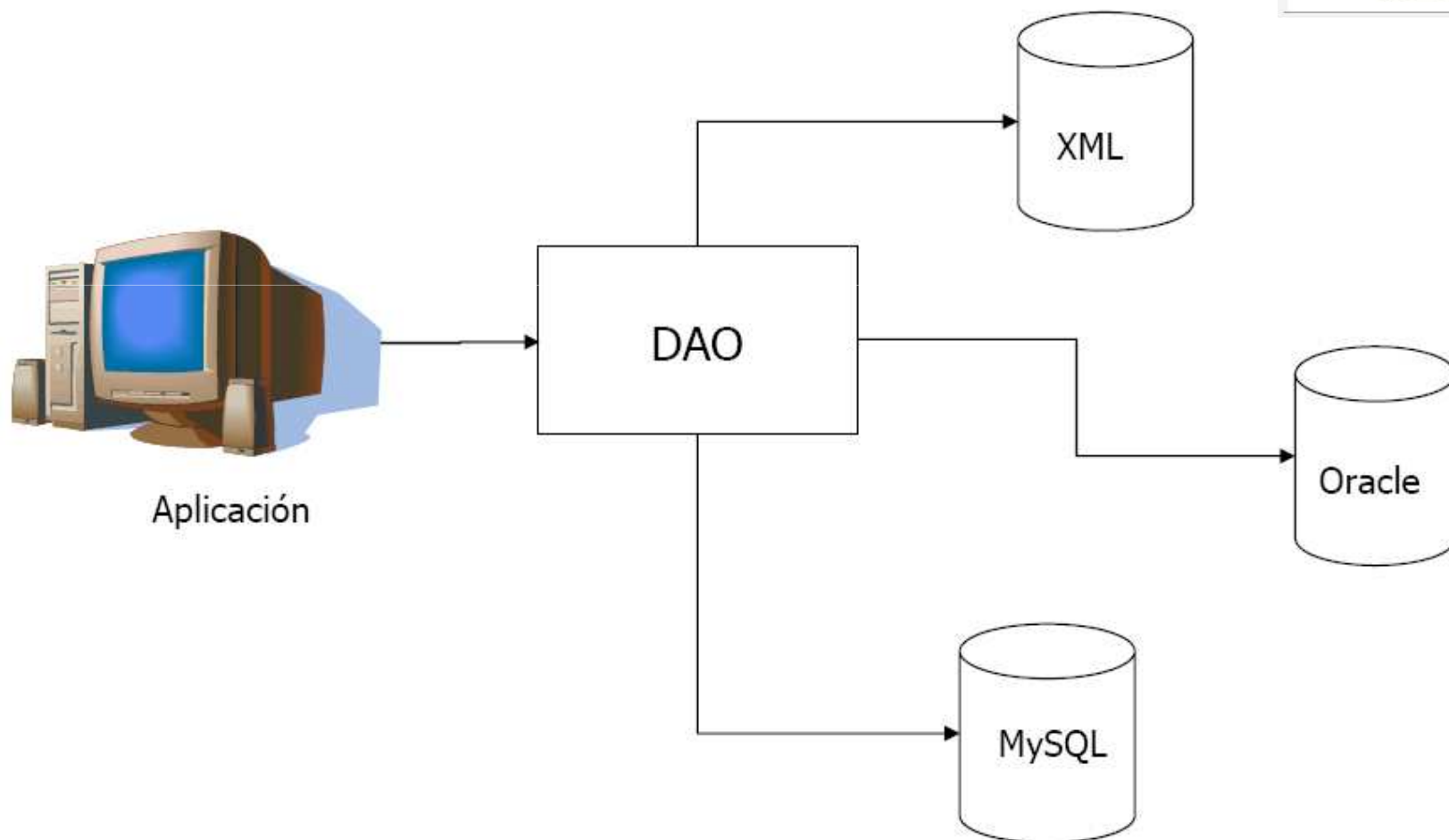
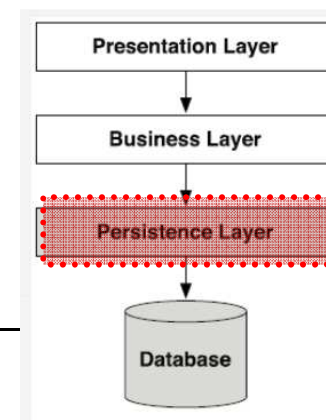


Patrón DAO: problemas si...

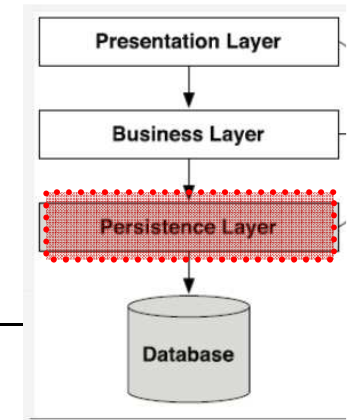


- ...Se necesita independencia del sistema de persistencia
 - BDD relacional, BDD orientada a objetos, ficheros, XML, BDD XML, serialización, ...
- ... Se debe acceder a varios sistemas desde la misma aplicación:
 - Y tienen APIs muy diferentes (o ligeramente)

DAO: solución

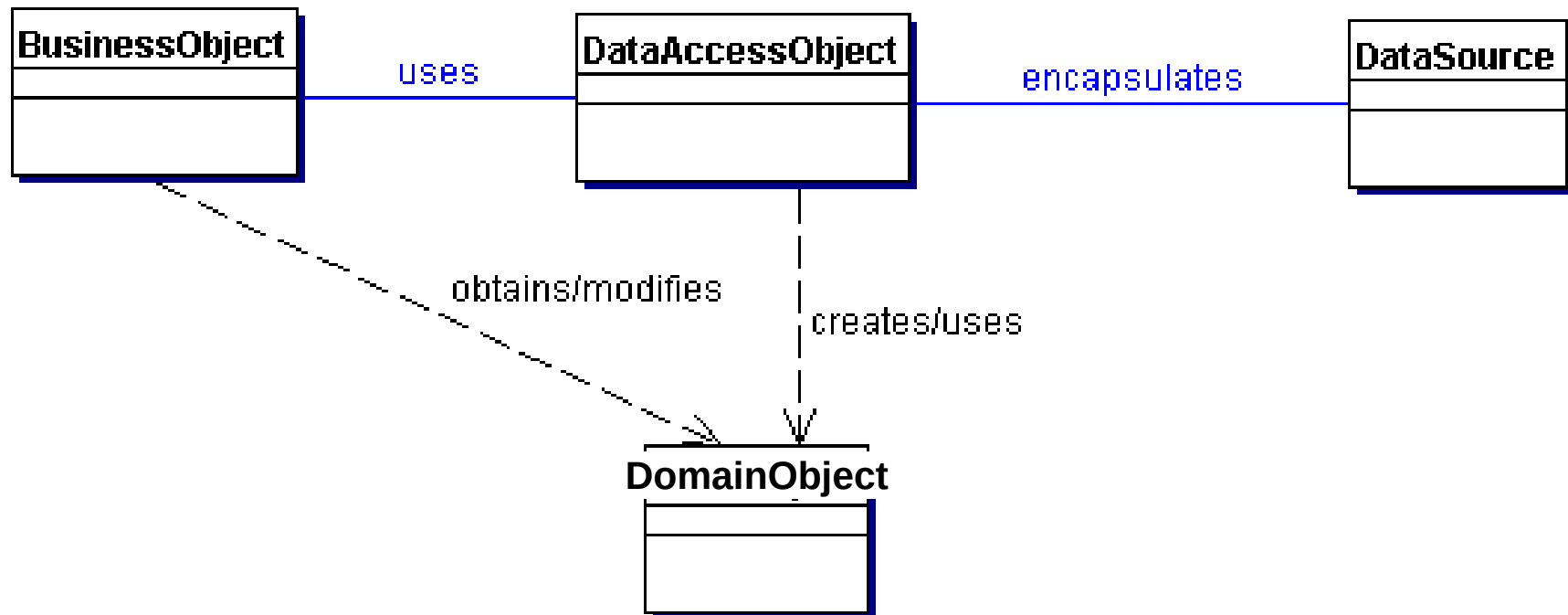
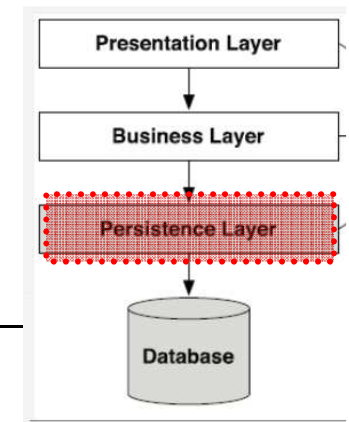


DAO



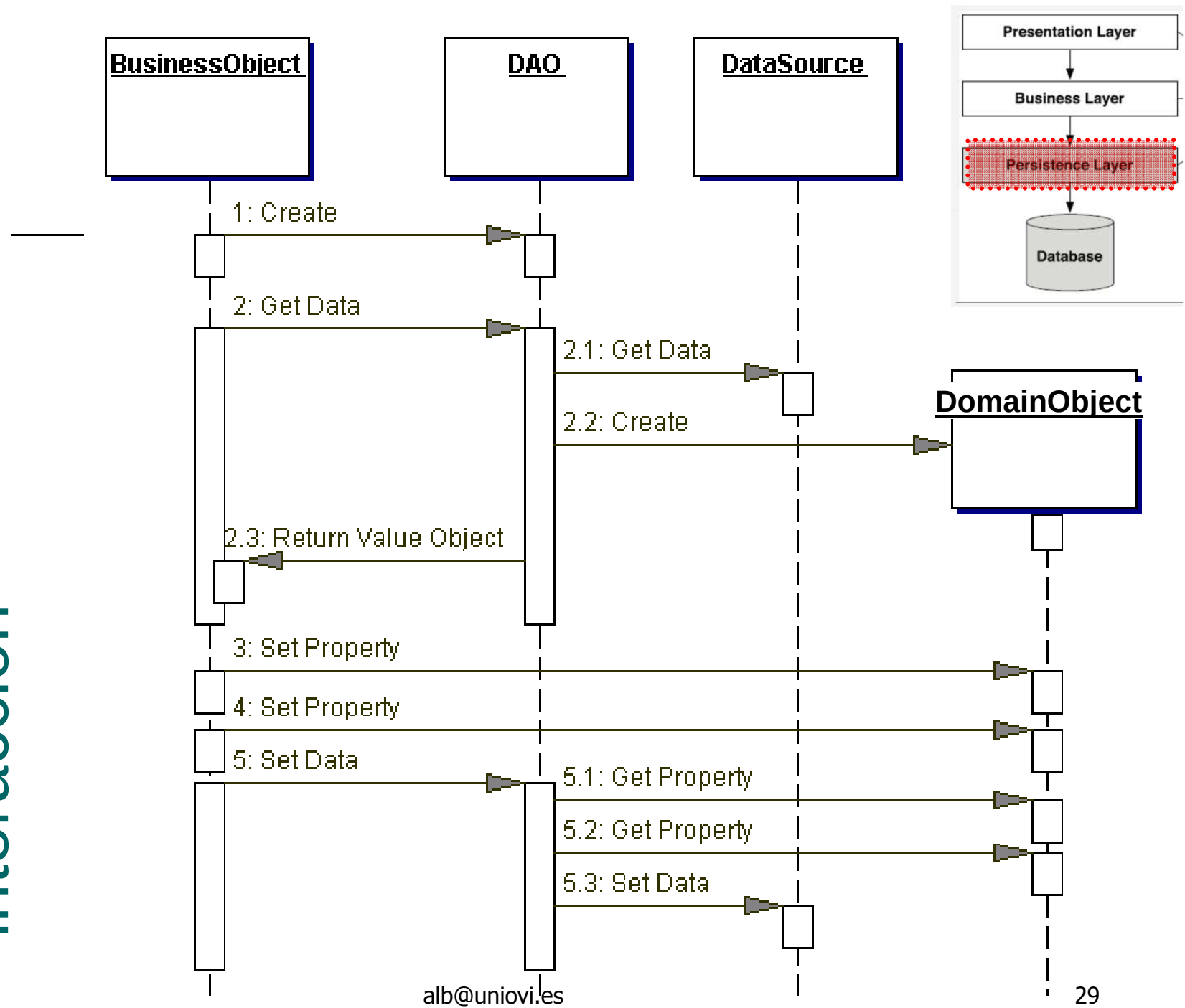
- DAO → Data Access Object
- DAO proporciona una interfaz única de acceso a los datos, de forma independiente a dónde se hallen almacenados.
- Independiza la lógica de negocio del acceso a los datos.
- Ofrece operaciones CRUD para cada objeto persistente del dominio

Modelo DAO



Modelo DAO: interacción

dic-08



alb@uniovi.es

29

Interfaces DAO: ejemplo

```
public interface GenericDao<T> {  
    void save(T t);  
    T update(T t);  
    void delete(T t);  
  
    T findById(Long id);  
    Collection<T> findAll();  
}
```

Métodos CRUD básicos

Métodos CRUD específicos
para cada entidad del modelo

```
public interface StudentDao extends GenericDao<Student>{  
    Collection<Student> findByNameSurname(String name, String surname);  
    Collection<Student> findByIdEnrollmentId(Long enrollmentId);  
    Collection<Student> findMatesByNameSurnameCourse(String name,  
        String surname, String course);  
    Set<Student> findUnenrolledAfterDate(Date date);  
}
```

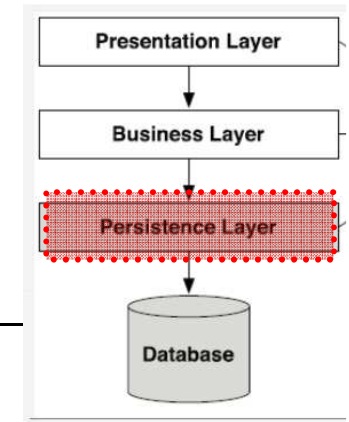


Código que
resuelve
lógica de
negocio →

```
public class ManageAuction {  
    ItemDAO itemDAO = new ItemDAO();  
    PaymentDAO paymentDAO = new PaymentDAO();  
  
    public void endAuction(Item item) {  
        // Reattach item  
        itemDAO.merge(item);  
  
        // Set winning bid  
        Bid winningBid = itemDAO.getMaxBid( item.getId() );  
        item.setSuccessfulBid(winningBid);  
        item.setBuyer( winningBid.getBidder() );  
  
        // Charge seller  
        Payment payment = new Payment(item);  
        paymentDAO.persist(payment);  
  
        // Notify seller and winner  
        ...  
    }  
    ...  
}
```

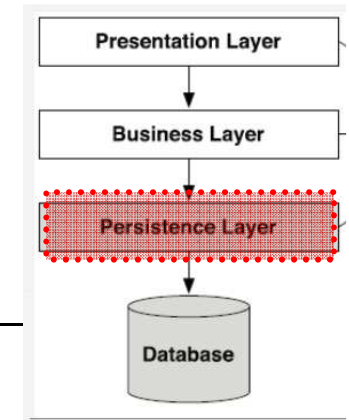
No tiene
dependencias de
persistencia

Posibles alternativas en JEE



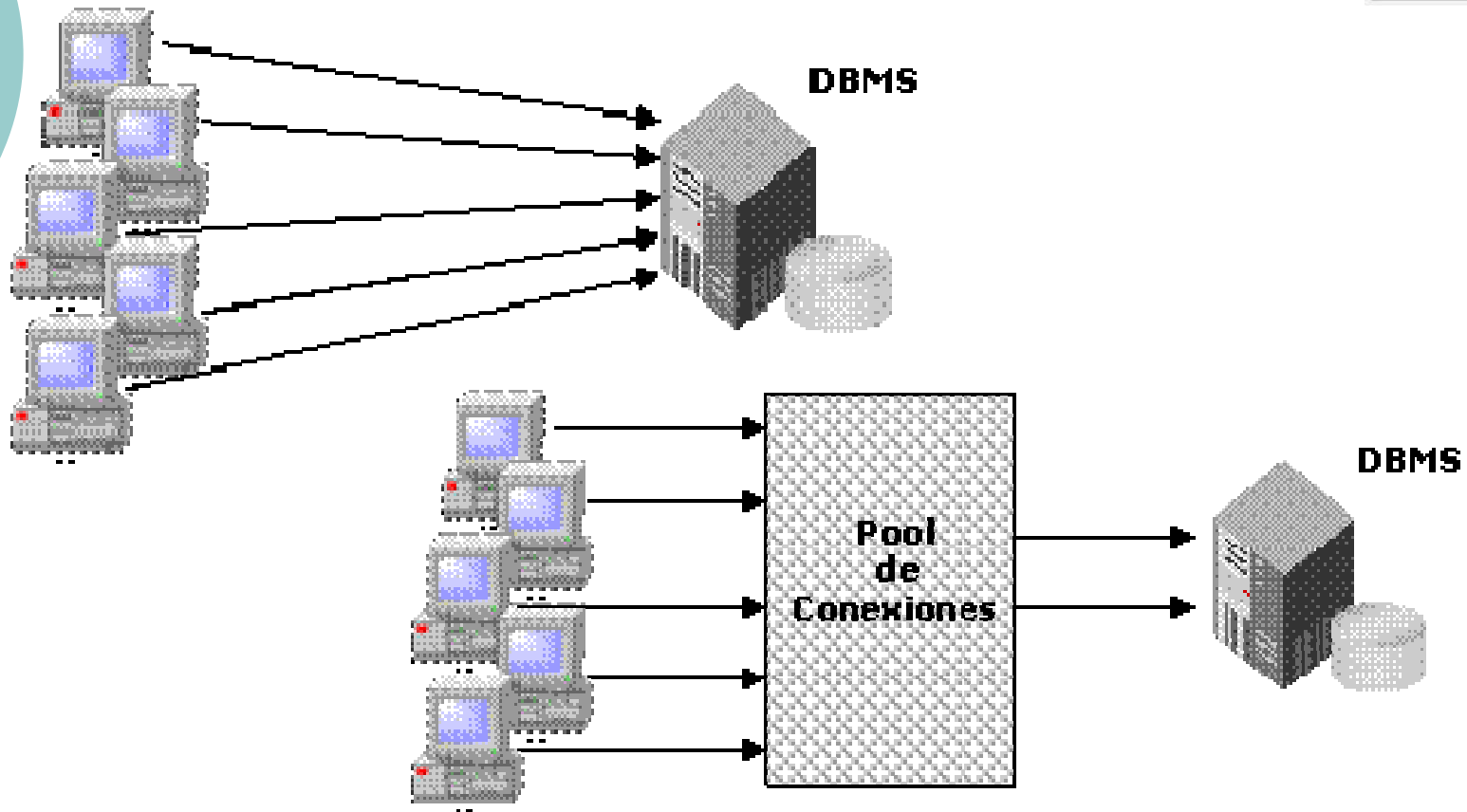
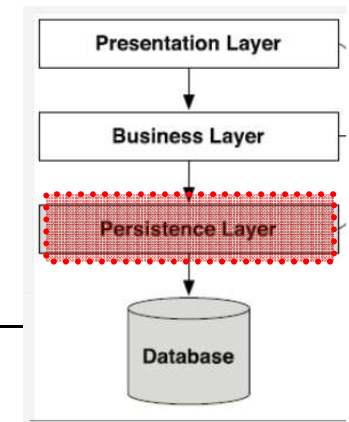
- Beans normales de acceso a datos
 - DAO con JDBC y SQL
- Framework de persistencia mapeo O/R
 - Hibernate, TopLink → EJB 3.0 JPA
- Conjunto de conectores a otros sistemas BackEnd
 - Ej. JCO o Business Connector para el acceso a SAP y BIW.
 - Integración con sistemas LEGACY
- Soluciones Híbridas de las anteriores.
- Generación de código JDBC

Pool de conexiones

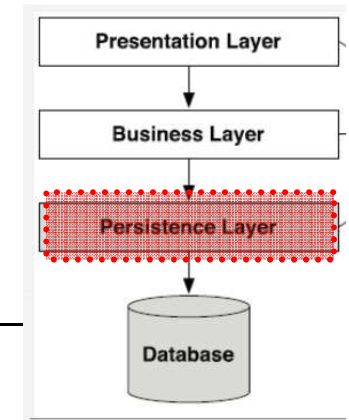


- Técnica destinada a gestionar y reutilizar objetos.
- Crear **conexiones** es una de las operaciones más caras → Abrir y cerrar conexión por cada consulta es muy costoso.
- Funcionamiento pool conexiones estándar:
 1. El cliente obtiene una referencia al pool
 2. Obtiene del pool una conexión abierta
 3. Ejecuta la sentencia SQL
 4. Devuelve la conexión al pool para que sea

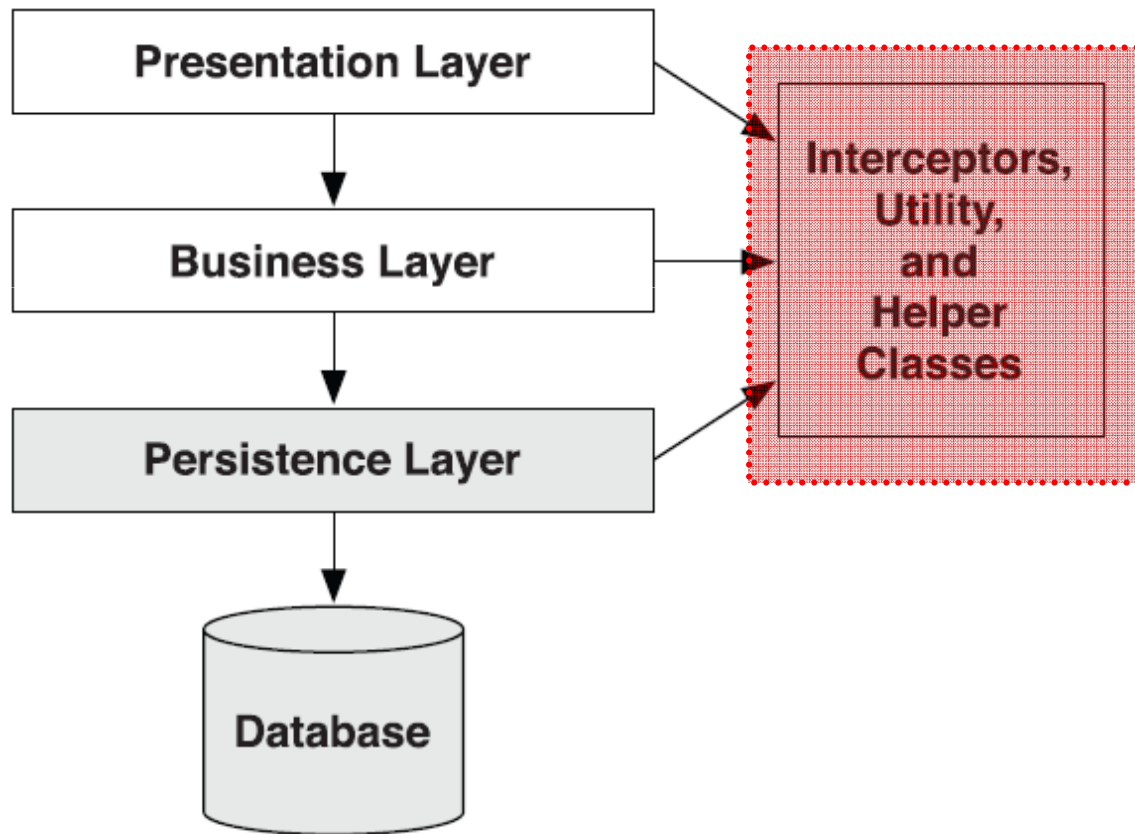
Pool de conexiones



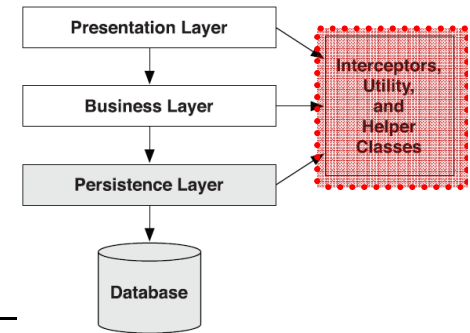
Externalización de SQLs



- Cada proveedor ha personalizado el SQL a su propia base de datos
 - Sentencias básicas son iguales, las optimizadas NO
- Consecuencia: Las SQLs limitan la portabilidad del sistema.
- Posibilidades:
 - Externalizar SQL a ficheroXML.
 - Se limita el impacto de una posible migración de base de datos a tareas de configuración
 - Permite hacer tuning mediante creación de índices, alteración del modelo, etc. sin necesidad de desarrollar.

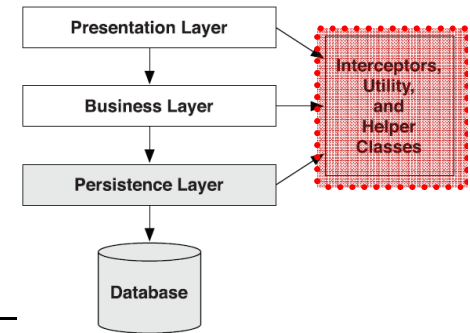


Capa de Infraestructura



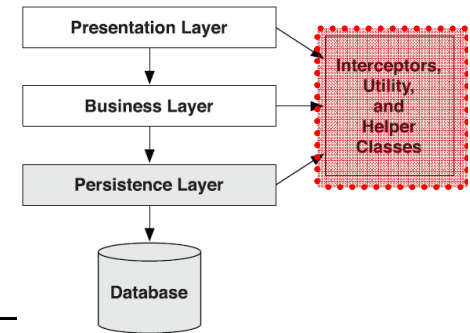
- Adyacente a todas las demás.
- Comprende todos aquellos servicios susceptibles de ser requeridos desde cualquiera de las capas lógicas del sistema.
- El servicio es un **componente** que suele ser dependiente del entorno de despliegue del sistema → **¿Portabilidad?**
 - Ej.: Servicio de Log varía de formato de salida de una empresa a otra, inclusive dentro del mismo grupo empresarial.

Servicios de Infra-estructura Habituales

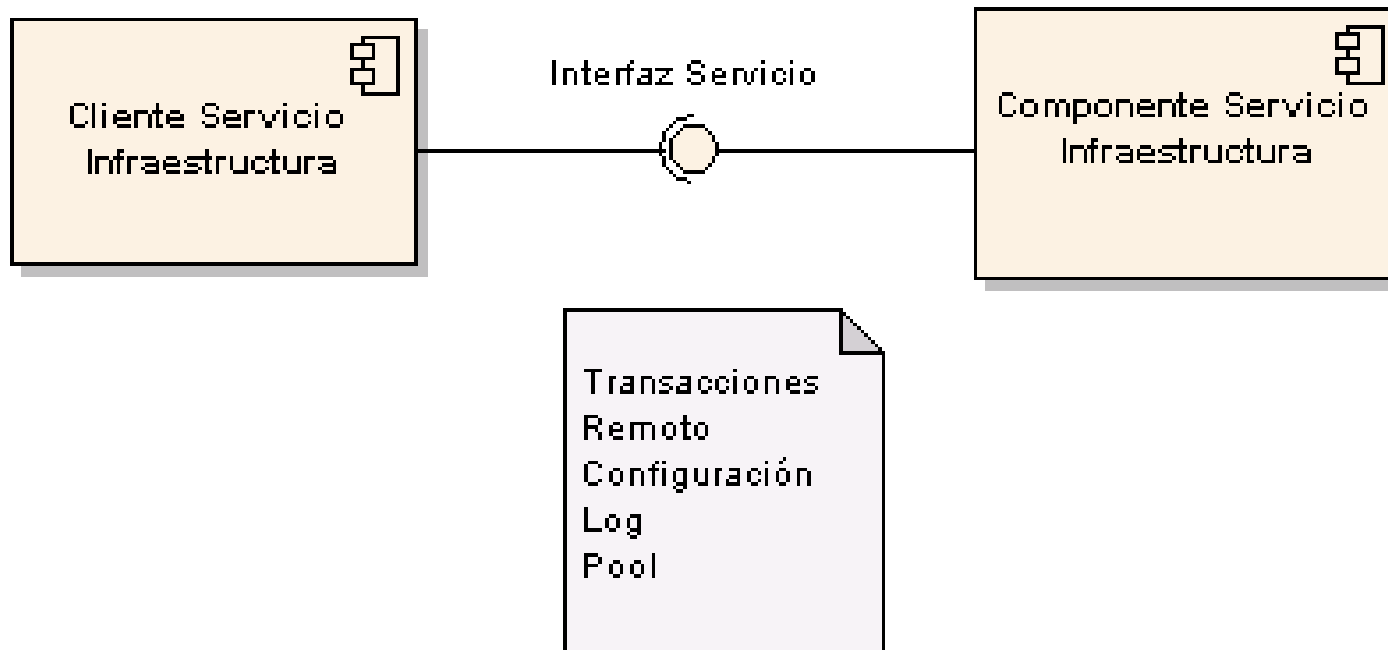


- Control de transacciones
- Servicio de log (logging != login)
- Pool de conexiones JDBC (o de cualquier otro sistema de persistencia)
- Sistema de configuración de la aplicación (Assembling)
- Gestor de accesos/permisos de usuario a los distintos servicios de la aplicación
- Provider de acceso a datos
- Otros...

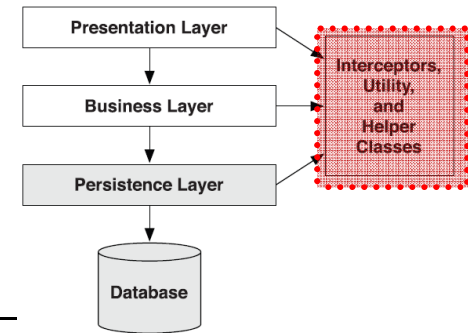
Gestión de los Servicios de Infraestructura



- Componentizados
- Se accede a ellos a través de una interfaz que define el servicio = contrato.

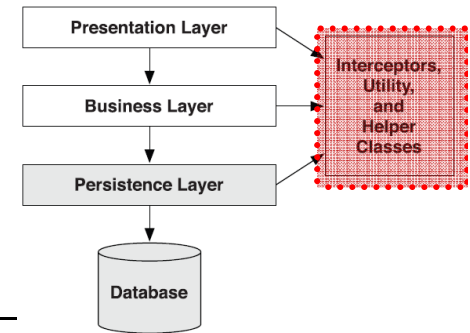


Gestión de los Servicios de Infraestructura



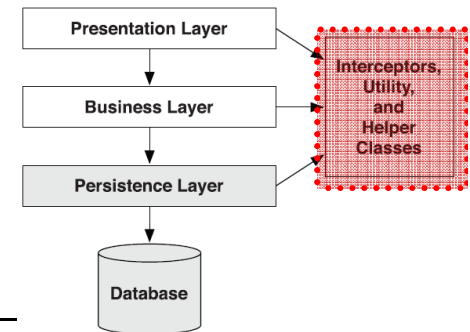
- La responsabilidad de instanciar la clase que sirve el servicio es de las clases gestoras.
- La relación de qué clase implementa un determinado servicio (interfaz java) en un momento dado se externaliza a un fichero XML
 - Cambios de infraestructura → reconfigurar XML
- Las clases del modelo no interactúan nunca con una clase de servicio de infraestructura directamente → siempre a través de la interfaz

Objetivos de la Capa de Infraestructura

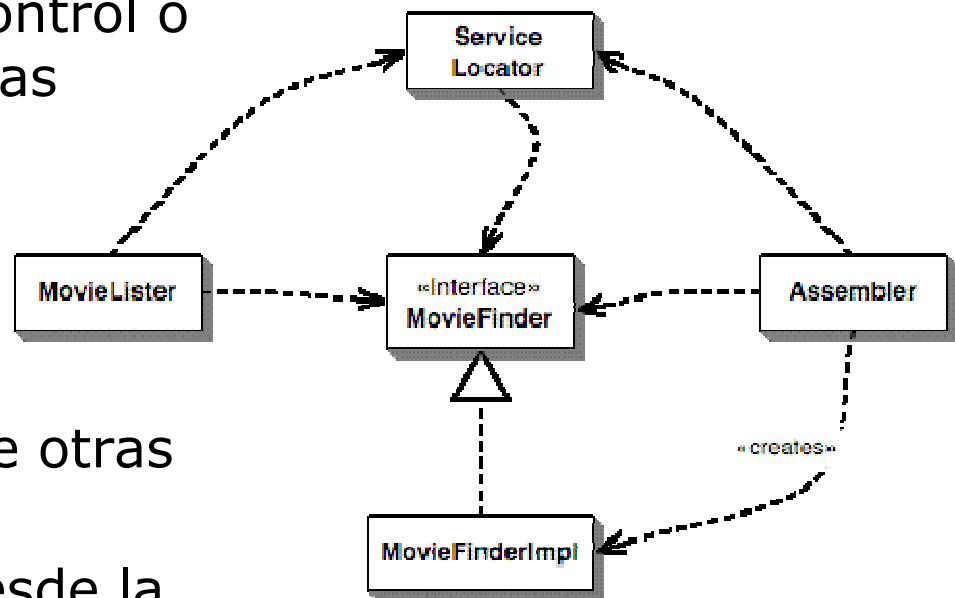


- Se desacopla completamente la aplicación del entorno de despliegue
 - La sustitución de un componente se limita a tareas de configuración
 - Clases de negocio usan interfaces de servicio
- Las clases gestoras pueden trabajar (en caso de que el componente lo permita) con pools de componentes
→ Aumento de rendimiento

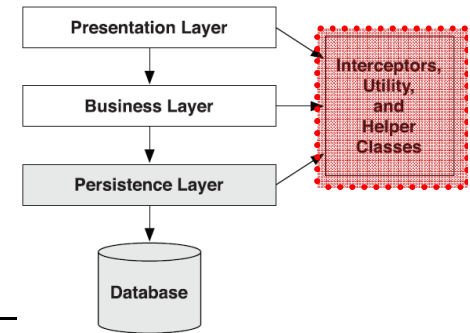
Frameworks IoC



- El patrón Inversion Of Control o Inyección de dependencias (Fowler).
- Apache Avalon/Excálibur
 - Ex Proyecto Jakarta.
- Spring Framework
 - Contenedor IoC (entre otras muchas cosas...)
 - Incorporado al FPA desde la versión 1.3
- PicoContainer
- EJB 3.0



Spring Framework



- Contenedor IoC:
 - Configuración centralizada y automatizada
 - *Cableado de beans*
 - No invasivo
 - Ensambla POJOs.
- Capa de abstracción para plugin de monitores transaccionales
- Capa de abstracción JDBC
- Integración con TopLink, Hibernate, JDO, e IBATIS
- Funcionalidad AOP
- MVC

Spring, externalización a xml de la configuración

```
<beans>
  <bean id="MovieLister" class="spring.MovieLister">
    <property name="finder">
      <ref local="MovieFinder"/>
    </property>
  </bean>
  <bean id="MovieFinder" class="spring.ColonMovieFinder">
    <property name="filename">
      <value>movies1.txt</value>
    </property>
  </bean>
</beans>
```

```
class MovieLister...
  private MovieFinder finder;
  public void setFinder(MovieFinder finder) {
    this.finder = finder;
  }
```

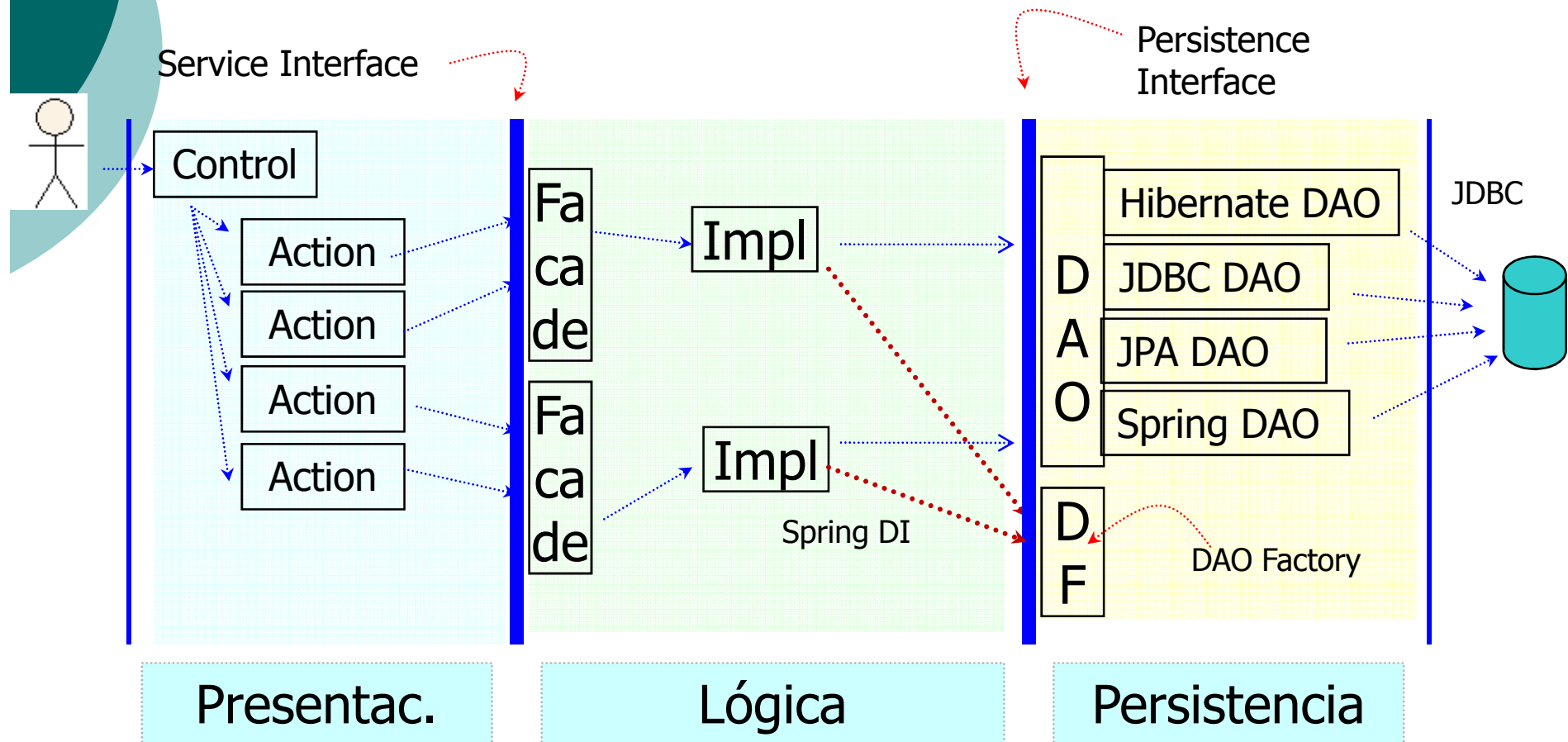
```
public void testWithSpring() throws Exception {
  ApplicationContext ctx = new FileSystemXmlApplicationContext("spring.xml");
  MovieLister lister = (MovieLister) ctx.getBean("MovieLister");
  Movie[] movies = lister.moviesDirectedBy("Sergio Leone");
  assertEquals("Once Upon a Time in the West", movies[0].getTitle());
}
```



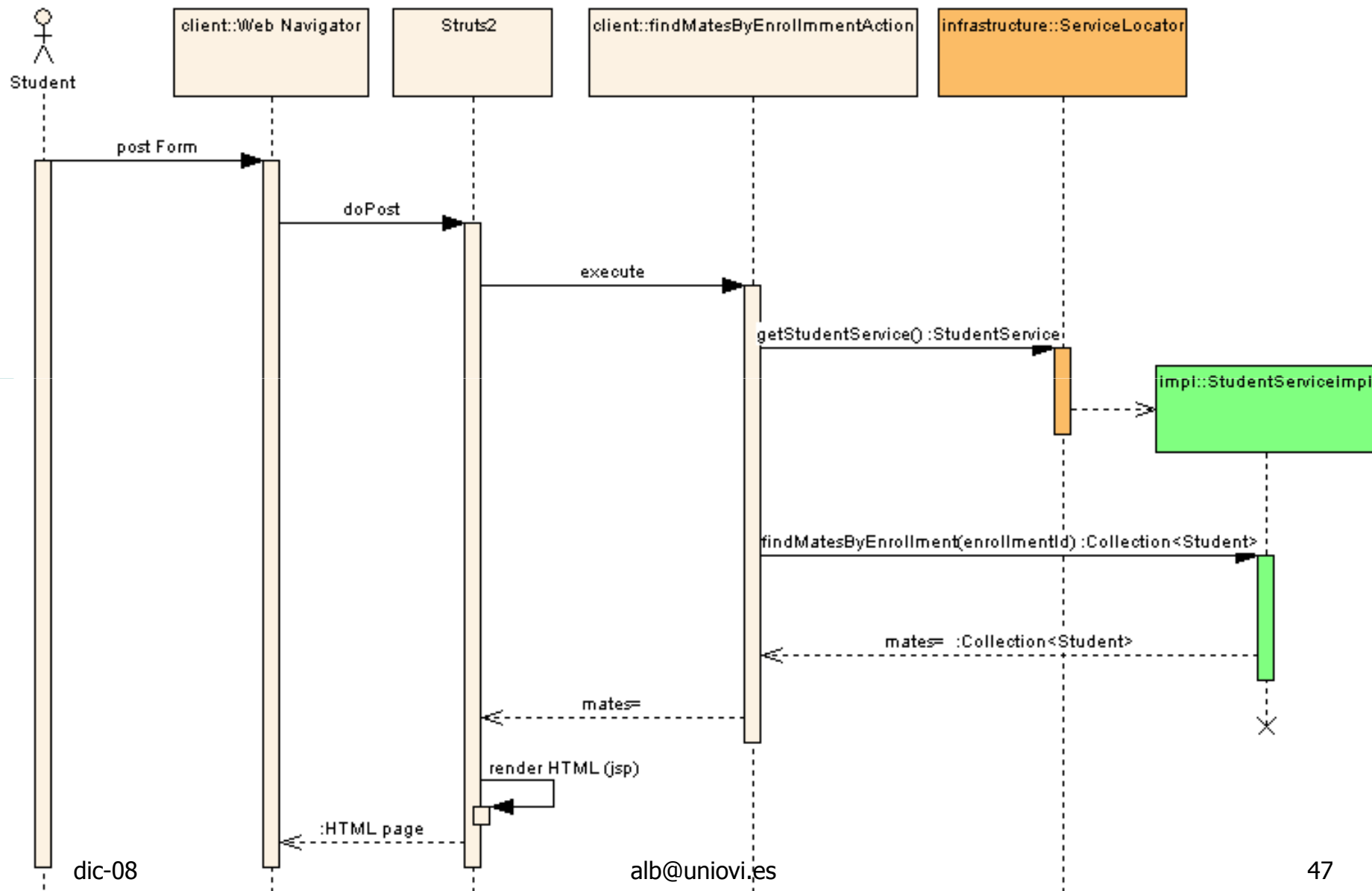
Arquitectura en capas: patrones

Presentación	Lógica	Persistencia
MVC	Fachada Comando Factoría	DAO Factoría

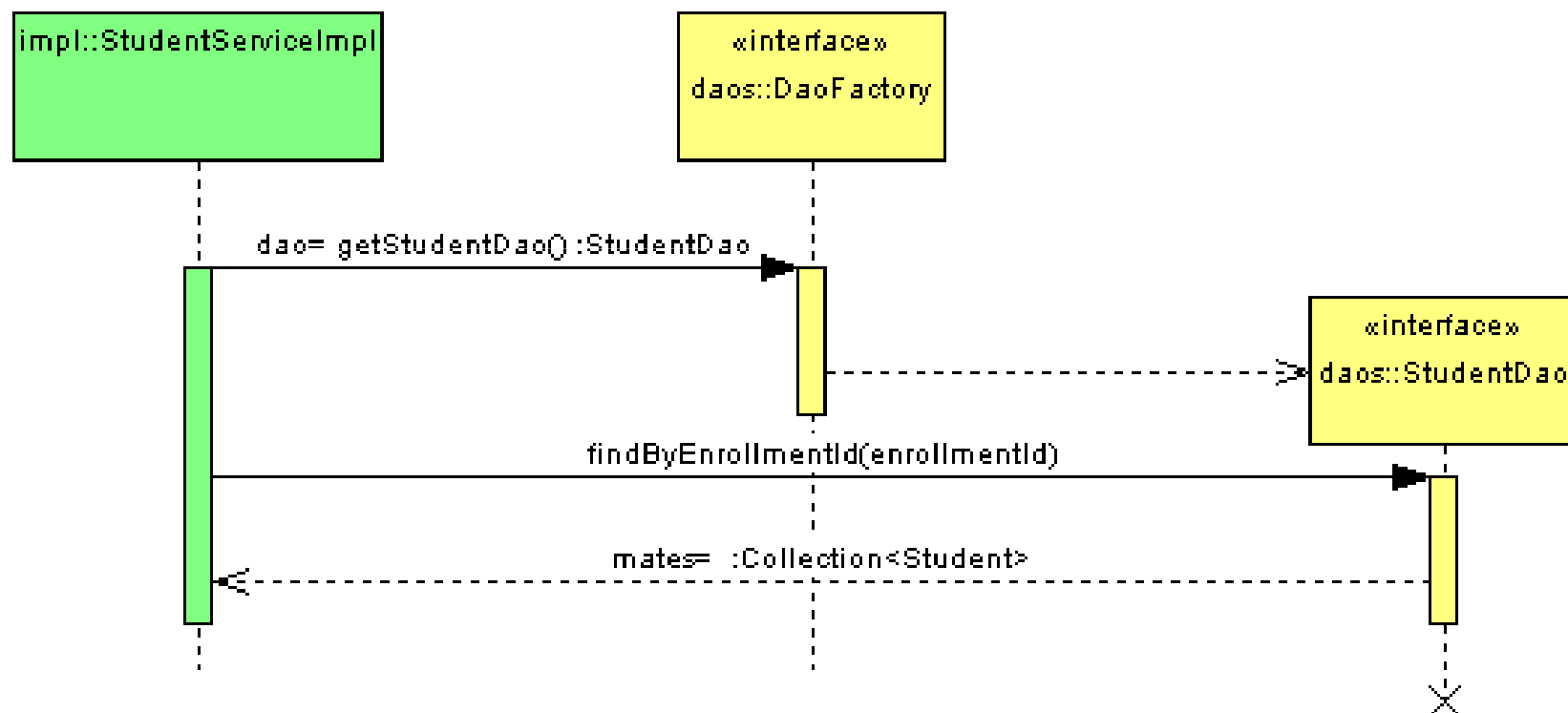
Solución en capas



Interacción entre las capas: ejemplo



Interacción entre capas: ejemplo lógica → persistencia





Referencias

- URLs
 - <http://jakarta.apache.org/Struts>
 - <http://theserverside.com>
- Libros
 - *Programming Jakarta Struts* de O'Reilly
 - *Mastering Tomcat Development* de WILEY
 - *Java Server Programming J2EE Edition* de Wrox